

1. DECREASE-AND-CONQUER

CORE STRATEGY

Solve a problem by reducing it to **one smaller instance**, solving it, and extending the smaller solution back to the original input. Exactly one recursive or iterative subproblem remains per step.

COMMON FORMS & EXAMPLES

- Decrease by 1:** Selection sort, insertion sort, linear search, topological sort (vertex removal).
- Decrease by k:** Batch maximum, counting, partial sum, block equality check, k-step search.
- Decrease by a Factor:** Binary search, fast exponentiation, tournament maximum, interval halving, power-of-two check.

CORRECTNESS & INVARIANTS

Preserved by local invariants assuming equivalent subproblem reduction:

Algorithm	Local Invariant
Selection Sort	Left prefix is completely sorted after each pass.
Linear Search	Every skipped element has been checked & is not the key.
Topological Sort	Every removed vertex has in-degree 0; safe to append next.
Binary Search	Target (if it exists) stays inside the retained half.
Fast Exponentiation	Exponentiation identity reuses results of smaller power.

COMPLEXITY & RECURRENCES

- Decrease by 1:** $T(n) = T(n-1) + f(n) \rightarrow$ Selection Sort: $O(n^2)$ time, $O(1)$ space. Linear Search: $O(n)$ time, $O(1)$ space. Topo Sort: $O(V + E)$.
- Decrease by k:** $T(n) = T(n-k) + f(n)$
- Decrease by Half:** $T(n) = T(n/2) + f(n) \rightarrow$ Binary Search: $O(\log n)$ time, $O(1)$ iterative space. Fast Exponentiation: $O(\log n)$ time.

EDGE CASES TO WATCH

Empty input (immediate return/failure); Size 1 (base case); Already sorted input (Selection sort still runs full comparisons); Duplicate values (define match policy); Non-DAG graph (Topo sort fails, no 0-in-degree vertex); Odd-sized inputs (last block $< k$).

2. DIVIDE-AND-CONQUER

CORE STRATEGY

Split a problem into **multiple subproblems**, solve them recursively, and **combine** their answers. Distinct from decrease-and-conquer due to branching into multiple tracks.

3-STEP EXECUTION PIPELINE

- Divide** Partition the input into smaller subproblems.
- Conquer** Solve smaller subproblems recursively.
- Combine** Merge sub-results into the final global solution.

CORE INVARIANTS & PROOFS

Proven via induction. Correctness relies on proper partition + perfect merge:

- Merge Sort:** Each sub-half is completely sorted prior to merge step.
- Maximum Element:** Global max is $\max(\text{left_max}, \text{right_max})$.
- Count Occurrences:** Total count = sum of left and right counts.
- Closest Pair:** Only a specialized vertical strip can beat half-minima.

3. AVL TREES (SELF-BALANCING BST)

CORE PROPERTY & BALANCE FACTOR

A Binary Search Tree that maintains a strict worst-case logarithmic height constraint via local balancing rules. The **Balance Factor (BF)** at every single node must satisfy:

$$BF = \text{height}(\text{left_subtree}) - \text{height}(\text{right_subtree})$$

Valid Balance Constraint: $BF \in \{-1, 0, 1\}$

REBALANCING MECHANISMS (ROTATIONS)

Rotations execute local pointer rearrangements to minimize height gaps. They **preserve the in-order BST properties** perfectly while altering structural depth.

Violation Type	Rotation Fix Required	Type
Left-Left (LL)	Single Right Rotation	$O(1)$
Right-Right (RR)	Single Left Rotation	$O(1)$
Left-Right (LR)	Double Rotation (Left then Right)	$O(1)$
Right-Left (RL)	Double Rotation (Right then Left)	$O(1)$

POST-MUTATION REPAIR PIPELINE

After any Insertion or Deletion action:

- Recalculate node heights upward.
- Evaluate localized Balance Factors.
- Apply rotation at the *first unbalanced ancestor*.
- Repeat process upward to the root if needed (cascading).

COMPLEXITY ANALYSIS

Operation	AVL (Balanced)	Plain BST (Unbalanced Worst-Case)
Search	$O(\log n)$	$O(n)$ (Degenerates to linked list)
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Rotation	$O(1)$	N/A

EDGE CASES & CRITICAL CONDITIONS

- Empty Tree:** Initial insert initializes the root node.
- Single Node Tree:** Default state; balance factor sits at exactly 0.
- Duplicate Keys:** Explicit pre-defined system-wide tree policy needed.
- Root Deletion / Chains:** Can trigger repeated, cascading upward rebalances across multiple parent layers.

4. FAST CORE CORE COMPARISONS

Decrease-and-Conquer	Tracks exactly one smaller subproblem instance.
Divide-and-Conquer	Branches into multiple smaller subproblems.
AVL Trees	Optimized for highly efficient ordered search operations.
Heaps	Optimized for priority access , not ordered search.
Selection Sort	Classified as a clear decrease-by-1 paradigm.
Binary Search	Classified as a clear decrease-by-a-factor paradigm.
Merge Sort	Classified as a pure divide-and-conquer paradigm.

- **Majority Element:** Global majority must survive local recursive matches.

COMPLEXITY & MASTER RECURRENCE

General Recurrence: $T(n) = aT(n/b) + f(n)$

(a = # of subproblems, n/b = subproblem size, $f(n)$ = divide/combine work)

- **Merge Sort:** $O(n \log n)$ time.
- **Closest Pair:** Standard D&C setup is $O(n \log n)$ time.
- **Max, Sum, Count:** $O(n \log n)$ if naively partitioned, or $O(n)$ if the combine step is linear and recursion remains shallow.

EDGE CASES TO WATCH

Empty array (return identity/error); Size 1 (standard base case); Odd-sized arrays (unbalanced halves); Duplicate values (impacts majority/counting); Non-unique answers (allow multiple valid outputs).

5. ONE-LINE EXAM CHECKLIST

- **I/O Profiles:** What exactly is the structured input and expected output?
- **Reduction Identity:** What exact smaller problem instance is created?
- **Invariant Integrity:** What invariant stays true after each progressive step?
- **Cost Models:** What is the exact recurrence formula or final runtime?
- **Boundary Triggers:** What are the base cases and edge cases?